

Homework Six : Trees

Question One

How many ancestors does a node at level n in a binary tree have? Provide justification.

A node of a binary tree at level n has n ancestors. This is because the root node is considered to be level 0, thus at level n you have a total height of $n+1$, inclusive of the node you're at, meaning $n + 1 - 1 = n$.

Question Two

Prove that a strictly binary tree (regular binary tree) with n leaves contains $2n-1$ nodes. Provide justification.

Given a strictly binary tree, we have two values which are involved in determining how many total nodes there are in the tree - the internal nodes and the leaf nodes. The number of leaf nodes is defined as internal + 1, which for the sake of the proof, we'll defined as $n = i + 1$. We also know that the total number of nodes in the tree is equal to $n + i$. So, using basic substitution, we can see that $n + (n-1) = \text{total}$ we find that the total $= 2n-1$, completing the proof through substitution

Question Three

Explain in detail that if m pointer fields are set aside in each node of a general m -ary tree to point to a maximum of m child nodes, and if the number of nodes in the tree is n , the number of null child pointer fields is $n*(m-1)+1$

So for the m -ary tree, each of the n nodes has m child pointers, meaning the total number of pointers in the tree is $m * n$. In the tree, there are going to be $n - 1$ child-parent relationships, and we know that the number of null pointers will be the total number of pointers minus the number of used pointers. This will leave us with the equation $\text{null_ptr} = m * n - (n-1)$, which can be factored to be $(m * n) - n + 1$, which we can pull n from givin $(m-1)n + 1$

Question Four

Define the Fibonacci binary tree of order n as follows: If $n=0$ or $n=1$, the tree consists of a single node. If $n>1$, the tree consists of a root, with the Fibonacci tree of order $n-1$ as the left subtree and the Fibonacci tree of order $n-2$ as the right subtree. Write a method that builds a Fibonacci binary tree of order n and returns a pointer to it.

we can pretty simply do this using recursion, with 0 and one as the base case.

```
struct tree_node node * right node * left end
```

```
function node * build_fib_tree(n); if(n == 0 || n == 1): return new tree_node;
```

```
else
    new node = new();
    root->left = build_tree(n-1)
```

```
root->right = build_tree(n-2)
return root;
```

endfunction: build_fib_tree

Question Five

Answer the following questions about Fibonacci binary tree defined in the previous problem. a. Is such a tree strictly binary? The tree is not strictly binary, as there are cases where nodes may only have one child, which would violate the strict binary condition of having two children or 0 b. What is the number of leaves in the Fibonacci tree of order n ? The number of leaves is just the fibonacci sequence, as the subtrees spawn in accordance with the sequence. Hence, $\text{num} = f(n)$ c. What is the depth of the Fibonacci tree of order n ? The depth is n , as the maximum depth will be defined by the number of times recursion is called, which is strictly associated with n